THE UNIVERSITY OF AZAD
JAMMU AND KASHMIR
DEPARTMENT OF SOFTWARE
ENGINEERING

LABORATORY EXPERIMENT REPORT · LAB TASK # 02

Dynamic Behavioral Modeling with UML 2.5: Activity & State Machine Diagrams

Student Name	**Syeda Khadija Hassan**
Roll Number	**2024-SE-09**
Degree & Session	BS Software Engineering (Session 2024–2028)
Semester	5th Semester (Fall ‘26)
Department	Department of Software Engineering
Course Title	Software Construction & Development (Lab)
Course Code	**SE-3101**
Instructor / Professor	Engr. Asma Javaid
Submission Date	October 24, 2026

1. Laboratory Overview & Objectives

This laboratory assignment focuses on the advanced dynamic behavioral modeling techniques specified in UML 2.5: Activity Diagrams with Partitions (Swimlanes) and State Machine Diagrams with rich internal activity semantics. While structural diagrams (e.g., Use Case, Class) define system boundaries and static configurations, behavioral diagrams capture the temporal evolution, data/control token transfers, state transitions, and asynchronous stimulus-response patterns fundamental to robust software construction.

> **[LEARNING OUTCOMES]**
> *Key Behavioral Modeling Mandates for Lab 2:*
> *1. Construct a multi-participant Activity Diagram using Swimlanes for the foodPanda mobile food delivery ecosystem, explicitly depicting concurrency (Fork/Join) and alternate decision flows.*
> *2. Develop a formal State Machine Diagram for an Electric Toaster, rigorously specifying entry, do, and exit activities within each state, while distinguishing Triggered events from Triggerless (anonymous/completion) transitions.*
> *3. Design an industrial-grade State Machine Diagram for a Passenger Elevator/Lift system, capturing motion scheduling, door interlock safety, obstacle detection, and emergency overrides.*

1.1 Activity Diagrams vs State Machine Diagrams

Understanding when and how to deploy Activity Diagrams versus State Machine Diagrams is critical in software engineering:

- **Activity Diagrams (Workflow-Centric):** Focuses on the procedural flow of control and data between distributed actors, subsystems, and business entities. Swimlanes partition responsibility, while decision diamonds and synchronization bars (fork/join) govern complex operational workflows.
- **State Machine Diagrams (Entity-Centric):** Focuses on the discrete operational modes and lifecycle states of a single entity (hardware controller, business transaction, or domain object). State transitions occur in response to explicit events (triggered) or automatic completion (triggerless), accompanied by entry, do, and exit actions.

2. Task 1: Activity Diagram for foodPanda Application (using Swimlanes)

2.1 System Context & Business Process Workflow

foodPanda is an on-demand multi-sided platform coordinating consumers, culinary vendors (restaurants), and independent logistics contractors (riders). A robust activity model must track the complete operational lifecycle from initial menu browsing to final food delivery and customer rating.

2.2 Swimlane / Partition Responsibility Matrix

Partition (Swimlane)	Operational Role	Key Activities & Decision Points
Customer / User	Initiator & Consumer	Browse menu, customize meal, add to cart, choose payment method (COD vs Online Card), confirm order, track rider, receive food, submit ratings.
foodPanda Platform	Digital Intermediary & Transaction Broker	Process digital card payments, validate discounts, assign order IDs, transmit orders to kitchen, execute rider dispatch algorithms,

		stream live GPS telemetry, log completed sales.
Restaurant / Kitchen	Order Fulfillment & Preparation	Receive incoming order alert, decide whether to accept/reject order, cook and package food, signal 'Ready for Pickup', hand over meal to rider.
Delivery Rider	Logistics & Delivery Execution	Receive broadcast dispatch, navigate to kitchen, pick up package, verify token, drive to customer address, collect COD payment (if applicable), hand over meal.

2.3 Decision Logic & Concurrency (Fork & Join) Analysis

- **Payment Gateway Decision:** The customer chooses between Online Payment and Cash on Delivery (COD). Online transactions are routed to the foodPanda payment gateway; upon failure, a retry prompt is issued. Successful card payments or COD selections converge to finalize order creation.
- **Kitchen Acceptance Decision:** Upon receiving the order, the merchant verifies kitchen capacity. If rejected (e.g., out of stock), the foodPanda system cancels the order and automatically issues an instant full refund.
- **Fork Synchronization (Parallel Execution):** Once accepted, the activity flow enters a Fork synchronization bar, splitting into two concurrent parallel threads: (1) the restaurant prepares and packages the food, while (2) the foodPanda system matches and dispatches a nearby delivery rider to travel to the restaurant.
- **Join Synchronization (Package Handover):** A Join synchronization bar ensures that the delivery journey cannot proceed until both concurrent branches finish: the restaurant marks the package 'Ready for Pickup' AND the rider arrives to inspect and collect the order.

2.4 Brief Introduction to the Activity Diagram

The diagram below depicts the end-to-end foodPanda order fulfillment process partitioned into four vertical swimlanes. The workflow originates in the Customer swimlane with an initial node (solid circle) and proceeds sequentially through menu browsing, meal customization, and checkout. Control tokens cross swimlane boundaries seamlessly: payment processing transitions into the foodPanda Platform partition, order alerts flow into the Restaurant partition, and dispatch triggers engage the Delivery Rider partition. Concurrency is rigorously modeled using UML fork and join synchronization bars, and alternative branches (card decline, restaurant rejection) ensure comprehensive exception handling until reaching the final activity node.

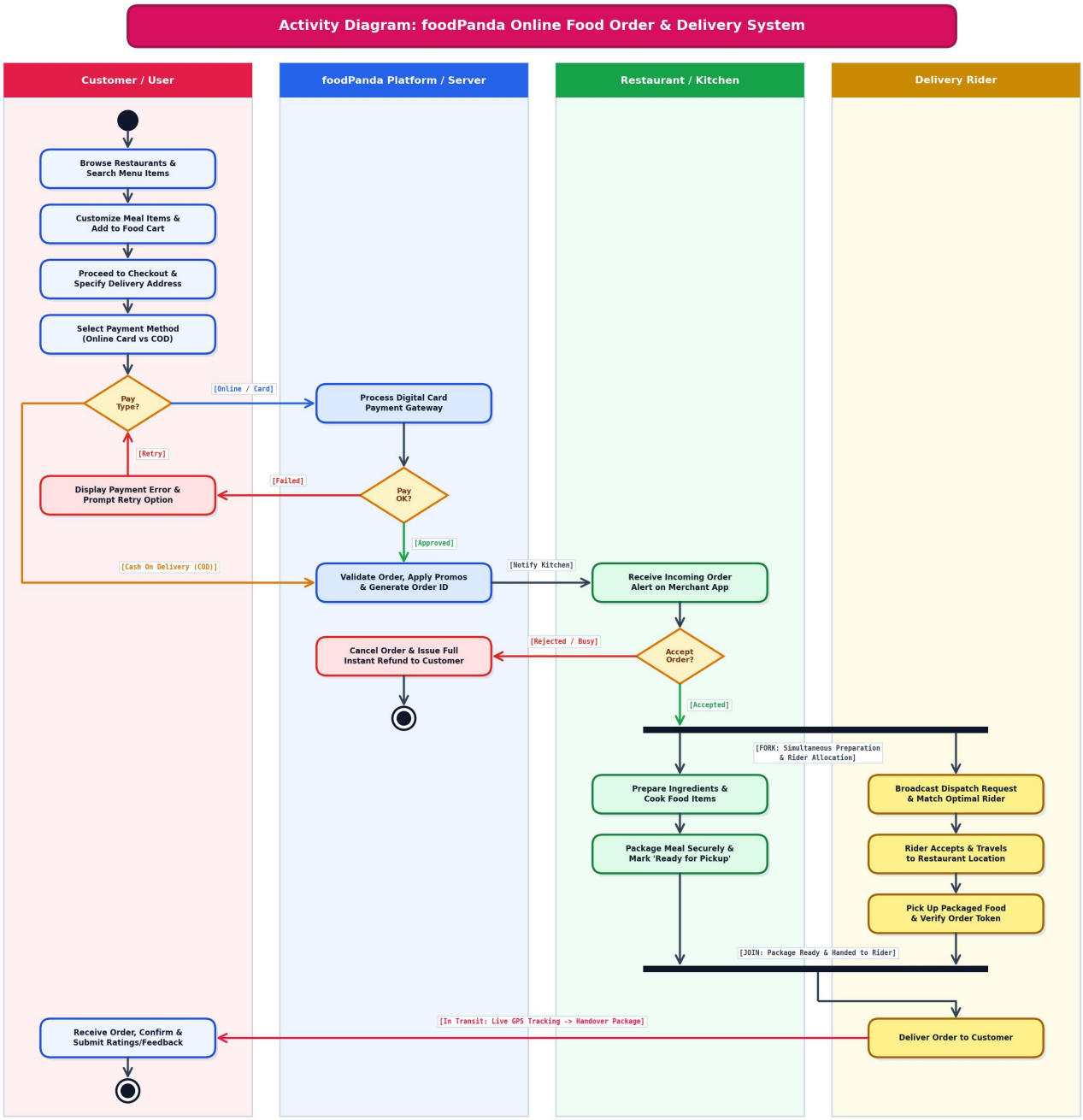


Figure 1: UML 2.5 Activity Diagram for foodPanda Online Food Order & Delivery System using Swimlanes.

3. Task 2: State Diagram for an Electric Toaster

3.1 Domain Context & Mechatronic Principles

An electric toaster is a classic embedded mechatronic appliance operating through discrete thermal and mechanical states. It integrates bread carriage slots, a spring-loaded mechanical lever with an electromagnetic solenoid latch, heating nichrome coils, a variable potentiometer browning control dial, bimetallic/thermistor sensors, and an audible piezoelectric chime.

3.2 Listing Activities Within Each State (entry, do, exit)

In strict compliance with UML 2.5 standards and the lab instructions, each state is defined by its internal activity compartments:

State Name	entry / Actions	do / Activities	exit / Actions
Idle (Ready)	displayReadyLed(ON) enableSlotSensors()	monitorSlotSensors() readBrowningDial()	prepareLatchingCircuit()
Bread Loaded	detectBreadPresence() playSoftClick()	monitorBrowningSetting() awaitLeverEngagement()	lockThermalProfile()
Toasting (Heating)	engageSolenoidLatch() energizeHeatingCoils() startBrowningTimer()	regulateInternalTemperature() decrementTimerCountdown() monitorInfraredSensor()	deEnergizeHeatingCoils() recordCycleTelemetry()
Cooling & Ejecting	releaseSolenoidLatch() popUpToast() soundBeepAlert(3)	dissipateResidualHeat() awaitBreadRemoval()	resetSensorsAndTimers()
Overheat Protection (Fault)	tripThermalFuse() cutHeatingPower() flashErrorLed(RED)	lockCarriageMechanism() monitorChamberCooling()	resetThermalCutoff()

3.3 Event Taxonomy: Triggered vs Triggerless (Completion) Events & Actions

The assignment specifically requires the explicit identification and contrast between Triggered and Triggerless events:

Classification	Event Signature	Source State	Target State	Action Executed
Triggered Event	insertBread()	Idle (Ready)	Bread Loaded	activateOpticalSensor()
Triggered Event	pushLeverDown() [powerOK]	Bread Loaded	Toasting (Heating)	latchCarriage(), startTimer()
Triggered Event	pressCancelButton()	Toasting (Heating)	Cooling & Ejecting	abortCycle(), cutPower()
Triggerless Transition	[timerExpired or browningAchieved]	Toasting (Heating)	Cooling & Ejecting	deEnergizeCoils() (Automatic)
Triggerless Transition	[toastRemoved and tempNormalized]	Cooling & Ejecting	Idle (Ready)	returnToReady() (Automatic)

3.4 Brief Introduction to the State Diagram

The following diagram presents the complete state space for the Electric Toaster appliance. Each state displays its formal UML 2.5 compartments, delineating the exact actions executed on entry, continuously during the state (do), and upon exit. The diagram clearly distinguishes between: (1) Triggered transitions (drawn in bold red and amber lines) that require explicit external inputs—such as a user inserting bread, pushing down the carriage lever, or hitting the cancel button; and (2) Triggerless completion transitions (drawn in forest green lines) that fire autonomously once internal timer or thermal conditions are met—such as automatic ejection when toast is browned, and automatic return to Idle once bread is removed and coils normalize.

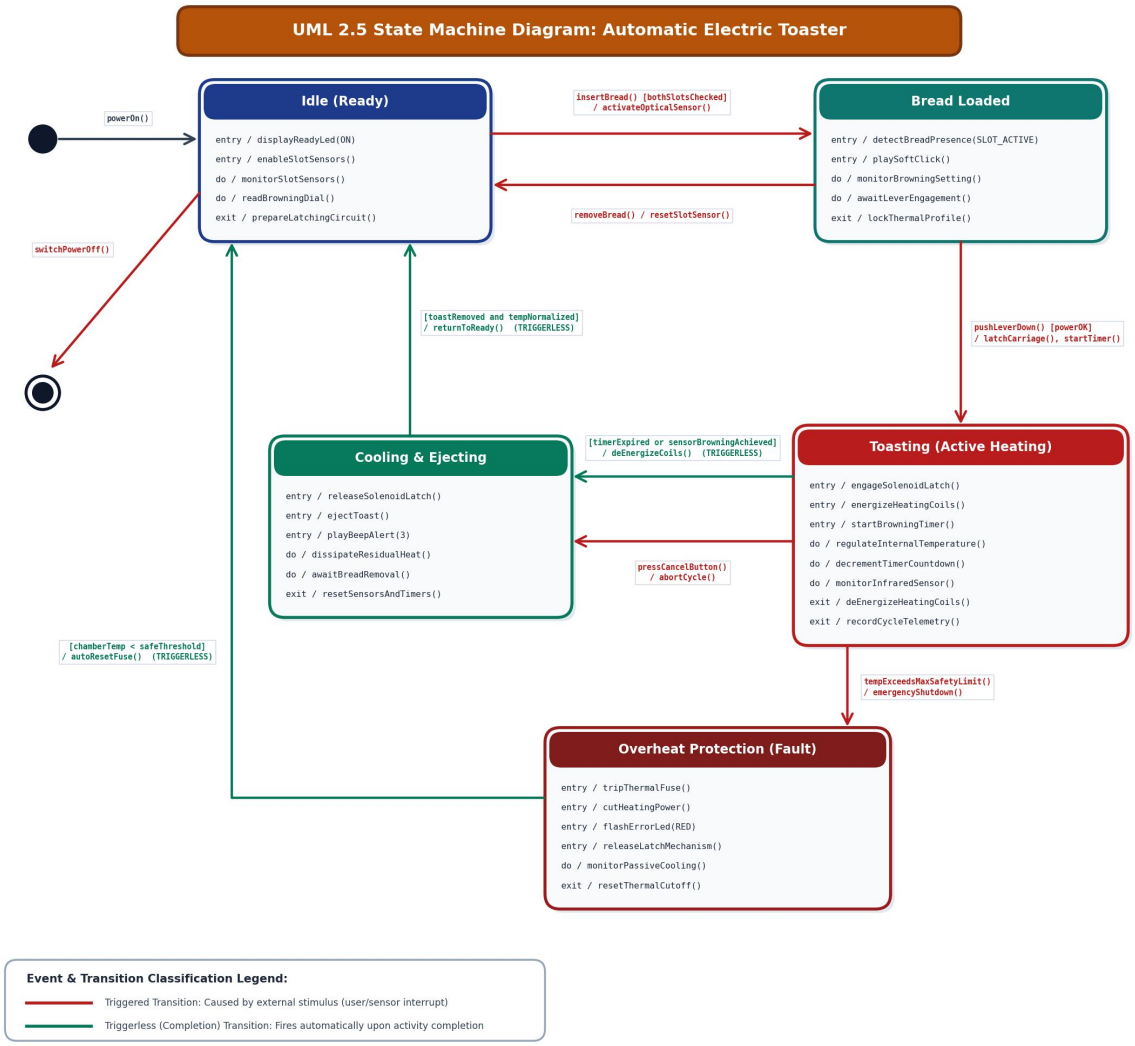


Figure 2: UML 2.5 State Machine Diagram for an Electric Toaster showing Internal Activities & Event Classification.

4. Task 3: State Machine Diagram of Elevator / Lift System

4.1 Industrial Elevator Control Architecture

Modern traction and hydraulic elevator systems represent safety-critical cyber-physical systems. The controller orchestrates high-torque hoisting motors, mechanical friction brakes, car and landing doors, hall and cabin call queues (using SCAN/LOOK collective algorithms), infrared light curtain sensors, load weighing scales, and emergency safety clamps.

4.2 State Transition Matrix & Control Logic

The operational dynamics are formalized in the following transition table:

Current State	Trigger / Event	Guard Condition	Action Executed	Next State
Idle at Floor	callRegistered	[targetFloor > currentFloor]	releaseBrake(), runMotor(UP)	**Moving Up**
Idle at Floor	callRegistered	[targetFloor < currentFloor]	releaseBrake(), runMotor(DOWN)	**Moving Down**
Moving Up/Down	targetFloorApproached	[levelSensorTriggered]	decelerateMotor(), clampBrake()	**Idle at Floor**
Idle at Floor	arrivedAtFloor	[carStationary and aligned]	activateDoorMotor(OPEN)	**Doors Opening**
Doors Opening	limitSwitchReached	[doorFullyOpen]	startDwellTimer(5s)	**Doors Open (Boarding)**
Doors Open	dwellTimerExpired	[noObstacle and weightOK]	activateDoorMotor(CLOSE)	**Doors Closing**
Doors Open	weightSensorTripped	[load > ratedCapacity]	soundBuzzer(), holdDoorsOpen()	**Overload Warning**

4.3 Safety Invariants & Closed-Loop Interlocks

- **Door Interlock Invariant:** The elevator car can NEVER transition to Moving Up or Moving Down unless the mechanical door interlock sensor confirms that all landing and car doors are hermetically locked.
- **Passenger Obstacle Reversal:** While in the Doors Closing state, an interruption of the optical light curtain or a depression of the 'Door Open' button causes an immediate motor reversal back into Doors Opening.
- **Overload Prevention Suspension:** If load cells detect weight exceeding rated limit (e.g., > 1000 kg), the system transitions to Overload Warning, sounds an acoustic buzzer, and holds the doors open until excess passengers step off.
- **Emergency Stop Mode:** Depression of the emergency red mushroom switch or power failure triggers immediate mechanical brake clamping, safety wedges drop onto guide rails, and emergency sirens activate.

4.4 Brief Introduction to the State Machine Diagram

The following diagram illustrates the complete reactive state machine for the Passenger Elevator/Lift controller. The central resting state is 'Stationary / Idle at Floor', wherein hoisting motors are disabled and mechanical holding brakes are locked. Directional motion is partitioned into 'Moving Up' and 'Moving Down', which evaluate scheduled floor calls via collective search heuristics. Upon arriving at the designated landing floor, level proximity sensors trigger deceleration and transition to the door control subsystem. The door lifecycle cycles through 'Doors Opening', 'Doors Open (Boarding)' with a 5-second dwelling timer, and 'Doors Closing'. Two dedicated safety loops protect passengers: an 'Overload Warning'

state that suspends elevator travel when weight limits are breached, and an 'Emergency Stop' state that can be activated from any motion phase to instantly freeze cabin movement.

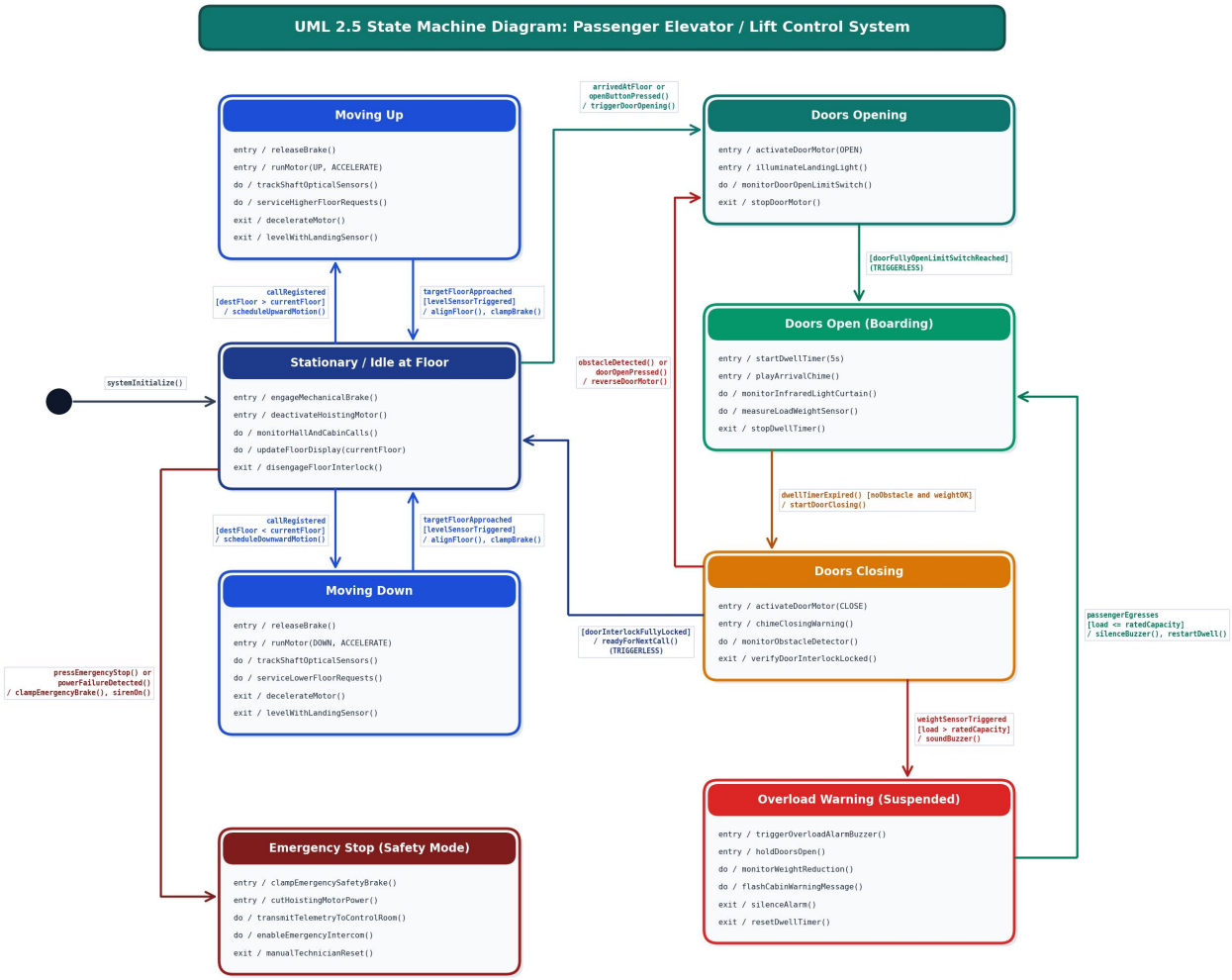


Figure 3: UML 2.5 State Machine Diagram for a Passenger Elevator / Lift Control System.

5. Comparative Synthesis: Behavioral Modeling in Software Construction

The exercises undertaken in this laboratory illustrate the complimentary nature of Activity and State Machine diagrams across software construction:

Modeling Dimension	Activity Diagram (Swimlanes)	State Machine Diagram
Primary Purpose	Models cross-system business workflows and computational step sequencing.	Models reactive event-driven lifecycles of an individual entity or hardware controller.
Unit of Abstraction	Action Node (a task or computational unit).	State (a condition or mode of existence during which invariants hold).
Concurrency Support	Native Fork and Join synchronization bars.	Orthogonal concurrent composite states or asynchronous event queues.
Transitions Driver	Completion of prior activity (token-driven).	Explicit external event triggers or internal

		state completions.
Ideal Construction Domain	Enterprise business systems, multi-agent platforms, microservice orchestrations.	Embedded firmware, real-time controllers, protocol engines, GUI lifecycles.

6. Conclusion

This laboratory report has comprehensively resolved all three tasks formulated by Engr. Saba Naseem for Lab 2 of Software Construction & Development. Through methodical domain analysis, an activity diagram with swimlanes was developed for foodPanda, visualizing complex multi-participant logistics and concurrency. For the Toaster appliance, an exhaustive state machine model was constructed, explicitly formalizing entry, do, and exit activities, and categorizing triggered versus triggerless events. Finally, an industrial elevator control model was engineered, providing robust closed-loop safety invariants and real-time state transitions. Together, these models provide a rigorous engineering specification ready for software implementation.